

AI個人開発してみようオフ会

2025.05.17 Sun @ Michikusa Office

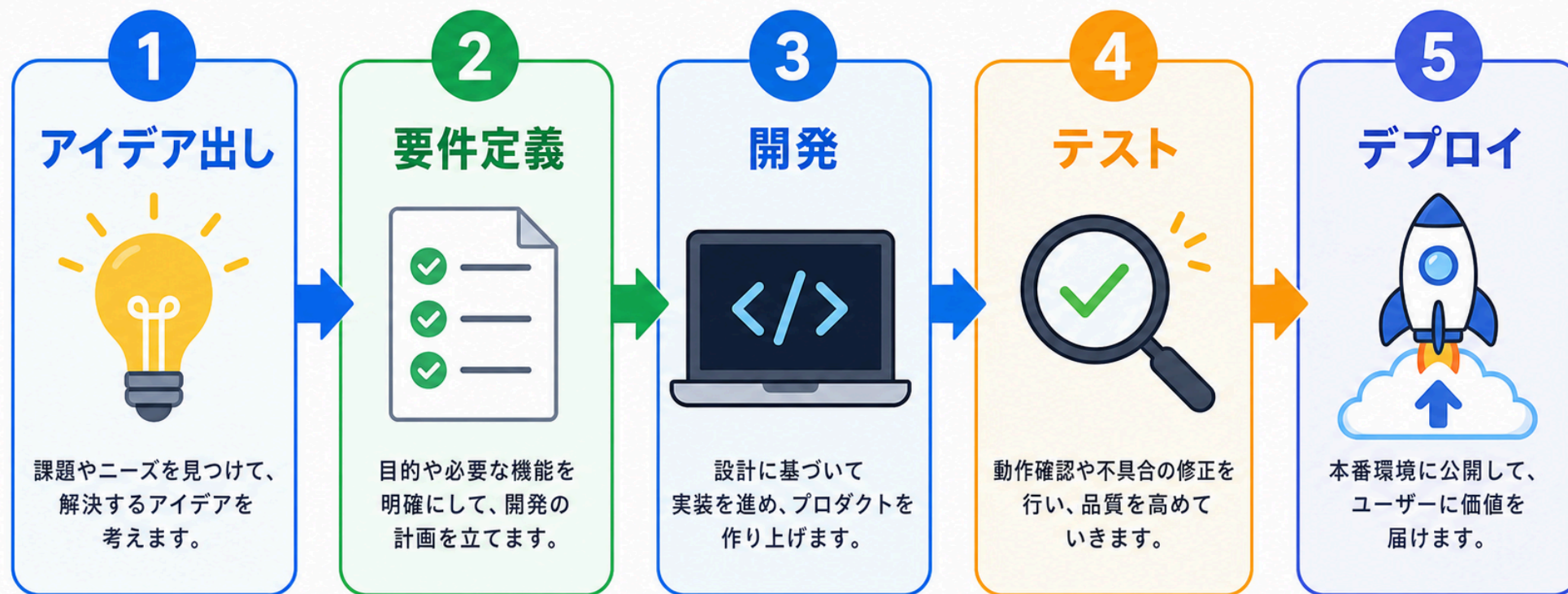
本日のスケジュール

時間	内容
13:15～13:45	説明 + ハンズオン
13:45～14:15	開発タイム・交流
14:15～14:45	デプロイタイム
14:45～16:00	開発タイム・交流

今日のゴール

個人開発する上で自分やユーザーを守るために最低限押さえておくべき考え方を知る

個人開発の流れ



個人開発する上で気をつけること

GitHubは基本プライベートで作る

GitHubはデフォルトでパブリックリポジトリとして作られるので注意。
基本的にプライベートリポジトリで作る。

種類	公開範囲
パブリック	全世界に公開される
プライベート	自分・指定した人のみ

プライベートにする理由

- 間違えて機密情報を入れても被害を最小限にできる
- アイデアや未完成の機能を守れる

プライベートだからといって何を書いても安全ではない。
APIキーやパスワードはプライベートリポジトリにも置かないのが基本。

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner *  kamekamek / **Repository name ***

[mokuyoukai_develop_0516](#) is available.

Great repository names are short and memorable. How about [shiny-happiness](#)?

Description

17 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository

Start with a template
Templates pre-configure your repository with files.

Add README
READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

Add license
Licenses explain how others can use your code. [About licenses](#)

☐ Public

☒ **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐ **Private**
You choose who can see and commit to this repository.

☐ No .gitignore

☐ No license

APIキー・シークレットキーについて

LINE・OpenAI・Slackなどの外部サービスと連携するときに発行される「サービスを使うための鍵」。

やってはいけないこと

- コードに直接書く（ハードコーディング）
- GitHubにアップロードする
- AIへの質問文に貼る

```
// 悪い例
const openaiApiKey = "sk-xxxxxxxxxxxxxxxxxx";

// よい例
const openaiApiKey = process.env.OPENAI_API_KEY;
```


APIキーは `.env` に入れる

```
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxx  
LINE_CHANNEL_ACCESS_TOKEN=xxxxxxxxxxxxxxxxxx  
SLACK_BOT_TOKEN=xoxb-xxxxxxxxxxxxxxxxxx
```

`.env` はGitHubに上げない。`.gitignore` に追加する。

`.env`

`.env.example` : 環境変数のサンプル

項目名だけ書いてgitで管理する（値は書かない）

```
OPENAI_API_KEY=  
LINE_CHANNEL_ACCESS_TOKEN=  
SLACK_BOT_TOKEN=
```

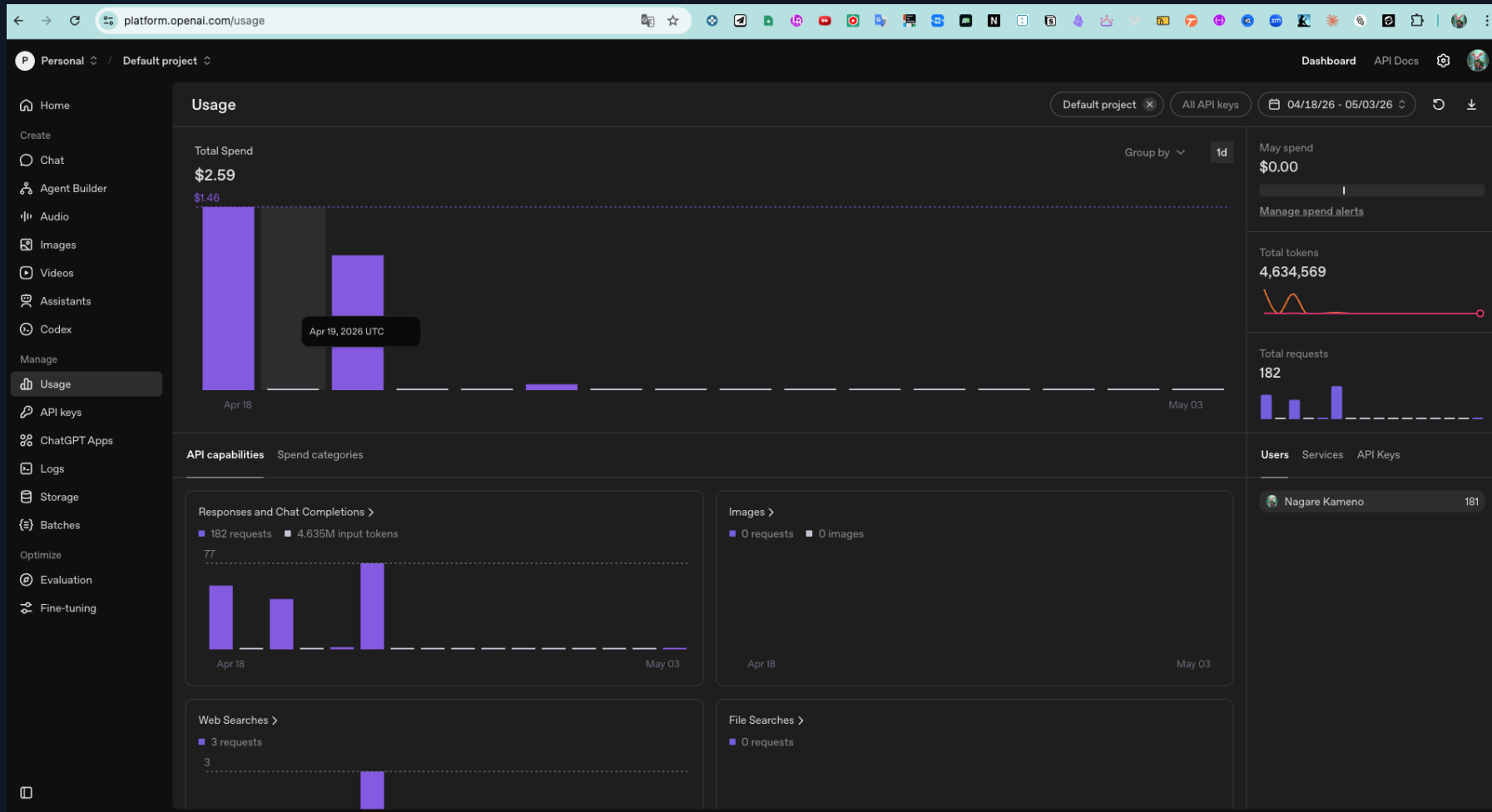
🐰 Vercelでデプロイする場合は「[Environment Variables](#)」で設定する必要がある

※APIキー等がコミットされていないか確認する方法として、`gitleaks` というツールがある（詳細は参考資料）

API keyの定期的な監視とローテーション

- API keyの利用上限を設定しておく
- 定期的に使用状況の確認（通知設定できると良い。）
- できれば、API Keyをローテーション（定期的な再発行）する

OpenAIのダッシュボード



<https://platform.openai.com/usage>

上限の設定(OpenAI)

The screenshot shows the 'Limits' section of the OpenAI platform settings. The left sidebar contains navigation links: Settings, Your profile, Organization, General, API keys, Admin keys, People, Projects, Billing, Limits (selected), Usage, Service health, Data controls, Security, Project, General, API keys, Webhooks, Evaluations, People, Limits, Cookbook, and Forum. The main content area is titled 'Limits' with a sub-tab 'Usage tier 2'. It features an 'Organization budget' section showing a May budget of \$0.00 / \$8.00 with a progress bar and a 'Resets in 28 days' note. Below this are two usage alerts: '37.5% usage alert' and '100% usage alert', each with a trash and edit icon. The 'Rate limits' section explains that API usage is subject to rate limits (TPM, RPM/RPD) and lists the organization's rate limits in a table. A note states: 'Note: Limits for specific model versions may vary, expand the table to see all models.'

MODEL	TOKEN LIMITS	REQUEST AND OTHER LIMITS	BATCH QUEUE LIMITS
gpt-5.1	1,000,000 TPM	5,000 RPM	1,350,000 TPD
gpt-5-mini	2,000,000 TPM	5,000 RPM	20,000,000 TPD
gpt-5-nano	2,000,000 TPM	5,000 RPM	20,000,000 TPD
gpt-4.1	450,000 TPM	5,000 RPM	1,350,000 TPD
gpt-4.1-mini	2,000,000 TPM	5,000 RPM	20,000,000 TPD
gpt-4.1-nano	2,000,000 TPM	5,000 RPM	20,000,000 TPD
o3	450,000 TPM	5,000 RPM	1,350,000 TPD

<https://platform.openai.com/settings/organization/limits>

キーをPushしてしまった場合

消しても履歴に残る可能性がある。「作り直す」が基本。

1. そのキーを無効化する
2. 新しいキーを発行する
3. `.env` やVercelの値を差し替える
4. 不審な利用や課金がないか確認する

AIに危険なコマンドを実行させない

コマンド	理由
<code>rm</code>	ファイルを完全に削除する
<code>rmdir</code>	ディレクトリを削除する
<code>chmod</code>	ファイル権限を変更する
<code>sudo</code>	管理者権限で実行する
<code>mv</code>	ファイル移動（上書きリスクあり）
<code>cp</code>	ファイルコピー（上書きリスクあり）

Macでは `.zshrc` に以下を追加するとゴミ箱に送れる。

```
alias rm = command trash "$@"
```

Claude Code の設定で制限する

`.claude/settings.json` のDenyリストに追加する。
完璧ではないが、防げるところは防いでおく。

```
{
  "permissions": {
    "deny": [
      "Bash(rm:*)",
      "Bash(rm -rf ~*)",
      "Bash(rm -rf .*)",
      "Bash(sudo:*)",
      "Bash(rmdir:*)"
    ]
  }
}
```


ハンズオン

Gitで戻れる状態を作る

Gitは変更履歴を残すための道具。

AIにコードを書かせると、一度にたくさんのファイルが変わることがある。

作業前にコミットしておくで、問題が起きたときに戻しやすくなる。

```
git add .  
git commit -m "作業前の状態を保存"
```

```
# 作業後の変更を確認する。  
git log --oneline
```

logの確認には

VSCodeの拡張機能「**Git Graph**」がおすすめ。

コミットの取り消し

Gitの変更は3段階を経る。`git reset` はどの段階まで巻き戻すかを指定するコマンド。

オプション	コミット	ステージ	作業ディレクトリ
<code>--soft</code>	取り消す	残る	残る
なし (<code>--mixed</code>)	取り消す	消える	残る
<code>--hard</code>	取り消す	消える	消える

基本は `--soft` を使う。変更内容が消えないのでやり直しがきく。

git reset の使い方

`HEAD` は「今いるコミット」を指す。`HEAD~1` は「1つ前のコミット」。

```
# 直前のコミットを取り消す  
git reset --soft HEAD~1
```

```
# 特定のコミットまで戻す  
git reset --soft abc1234
```

コミットIDは `git log --oneline` で確認できる。

現在地

時間	内容	
13:00～13:30	講義 + ハンズオン	✓
13:30～14:30	開発タイム・交流	👉
14:30～15:00	Vercel解説・作ったものの共有・クロージング	

開発タイム・交流

AIを使って何か作ってみよう。
詰まったら隣の人に聞いてみよう。

現在地

時間	内容	
13:00～13:30	講義 + ハンズオン	✓
13:30～14:30	開発タイム・交流	✓
14:30～15:00	Vercel解説・作ったもの共有・クロージング	👉

Vercelでデプロイする

- Pushすると自動でデプロイされる
- 環境変数をVercel側（[Environment Variables](#)）で設定する
- Claude CodeのVercelプラグインを使うと「デプロイして」だけで設定が完了する

ブランチ戦略

最初は `main` だけでよい。公開アプリになってきたら2つに分ける。

ブランチ	役割
<code>main</code>	本番用。常に動く状態を保つ
<code>develop</code>	開発用。AIと一緒に作業する場所

なぜ分けるのか

- 本番環境に影響を与えず、機能開発を進められる
- `develop` ブランチで一度デプロイして動作確認することで本番環境での予期せぬ不具合を事前に防ぐことができる

開発の流れ

1. `develop` ブランチを作成して開発する
2. `main` に対してPR(Pull Request)を作成する
3. PRの中で発行されたVercelのプレビューURLで確認する
4. 問題なければ `main` にマージする

新しいブランチの作り方

```
git checkout -b develop
```

checkout: ブランチの移動

-b : 新しいブランチを作ってそのブランチに移動する

VercelのプレビューURLを使う

VercelをGitHubと連携すると、ブランチ・PRごとにプレビューURLが作られる。

確認できること

- 画面が崩れていないか
- スマホでも見られるか
- AIが変えた内容が正しいか
- 本番に出してよい状態か

いきなり本番に出すのではなく、プレビューURLで確認してから本番に入れる。

作ったものの共有

作ったもの・開発中のもの・気づき、なんでもOK

まとめ

- GitHubはプライベートで。APIキーはコードに書かない
- `.env` に書いて `.gitignore` で除外、Vercelの環境変数に設定する
- 従量課金のAPIを使うときは利用上限を設定し、定期的に使用状況を確認する
- API Keyをushしてしまったら即無効化・再発行する
- AIに `rm` / `sudo` などの危険なコマンドを実行させない
- 作業前にコミットして「戻れる状態」を作る

補足資料

gitleaks でスキャンする

コミット履歴にAPIキー等が含まれていないかをスキャンするツール。

```
# スキャン
gitleaks detect

# 詳細表示
gitleaks detect --verbose
```

インストール (Mac)

```
brew install gitleaks
```

Homebrewのインストール (Mac / WSL共通)

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```


gitleaks (Windows)

1. <https://github.com/gitleaks/gitleaks/releases> にアクセス
2. 最新版の `gitleaks_x.x.x_windows_x64.exe` をダウンロード
3. 任意の場所（例： `C:\tools\gitleaks` ）に保存
4. そのフォルダを環境変数 PATH に追加
5. Git Bash を開いて疎通確認

```
gitleaks version
```

gitleaks の出力例

```
Finding:      OPENAI_API_KEY=sk-proj-aB3xK9mNpQ2...  
Secret:       sk-proj-aB3xK9mNpQ2...  
RuleID:       openai-api-key  
File:         .env  
Line:         1  
Commit:       a1b2c3d4e5f6...  
Author:       your-name  
Date:         2025-05-03T17:05:00Z
```

```
leaks found: 1
```

見つかったら該当のAPIキーをすぐ無効化して新しいキーを発行する。

要件定義の進め方

- AIにヒアリングしてもらいながら要件を固める
- UIをバイブコーディングで先に作り、後から要件を詰める方法もあり

ポイント

- 機能を盛り込みすぎない
- AIの回答（文章）をちゃんと読む
- 理解できないところは深掘りして、自分の言葉で残す

今から〇〇というアプリを作りたい。
一問一答でヒアリングして要件を固めてください。

AIの出力はMarkdownよりHTMLが強い

Markdownは読みやすいが、100行を超えると読まれにくい。

HTMLならAIが色・図・表・インタラクションを自由に表現できる。

Markdownでは難しいこと

Markdown	HTML
文字・箇条書きのみ	色・デザイン (CSS)
ASCII図が限界	図・フローチャート (SVG)
静的なテキスト	インタラクション (JavaScript)

HTMLで作れるもの

- 仕様書・実装計画
- PRのコードレビュー資料
- ビジュアルレポート・週次報告

「〇〇について調べてHTMLファイルにまとめて」

参考

- <https://zenn.dev/motowo/articles/ai-agent-html-output-design>
- <https://thariqs.github.io/html-effectiveness/>
- <https://x.com/trq212/status/2052809885763747935>

